21 — Date rules, and why they live where they do

A date entered on a form is checked in three places, and only one of them can actually stop a bad value. This document records what each layer can and cannot do, what the system does when a date breaks a rule, and the parts that remain incomplete.

It exists because the answer is not obvious, and because two of the three layers behave in ways a reader would not predict.

---

1. The three layers

Layer	What it is	Can it block a bad date?
The calendar in the form	a `datetime` component's picker	**only one direction** (see §2)
The shared rule table	`workers/src/main/resources/date-rules.json`, read by both the Java validator and the Python form tooling	yes, server-side
The BPMN model	the `INVALID_DATE` error and the boundary events that catch it	decides what *happens* after a rejection

---

2. What Camunda Forms can and cannot express

The `datetime` component offers **Disallow past dates** (`disallowPassedDates`), which sets the picker's minimum to today. It has **no counterpart for the future**: there is no "disallow future dates" property, and the component's `validate` object accepts only `required`. `validate.max` exists for `number` fields, not for `datetime`.

The missing feature is tracked upstream as camunda-modeler#3677 ("Add 'Disallow future dates' to Form Date input constraints"). Until it lands, a date field can only be constrained in **one** direction by the form itself.

Two consequences follow, and both are visible in the delivered forms:

1. A **future-date** field is enforced twice — the picker refuses earlier dates and the worker refuses them too. Eight fields are in this class.
2. A **past-only** field (a date recording something that has already happened) can only be enforced by the worker. Nine further fields are in this class. Nothing in the form stops a future date being picked.

`date-rules.json` carries the classification, so the two layers cannot drift apart.

2.1 A note on the field label

The label of a `datetime` field is **not rendered** by form-js. A datetime carrying `label: "Date of decision (today or earlier)"` renders its label area as the required asterisk alone, while every other control type renders its label normally. This was established by rendering the delivered forms through `@bpmn-io/form-js` under headless Chrome, not by inspection.

So the rule is stated in the two places a reader does see: a bold **text view** immediately above the date field, and the field **description** immediately below it. The label is kept in the form source because it still reads in the Camunda Modeler.

---

3. What happens when a date breaks a rule

A date is not a transient fault. Retrying cannot fix a value that a person typed, so the violation is raised as the **BPMN error** `INVALID_DATE` rather than as a failed job.

```
user task (form)  ──►  service task  ──►  INVALID_DATE boundary  ──►  same user task
                          │                                            (correct and resubmit)
                          └─ date rules hold ──► the pathway continues
```

Thirteen boundary events carry that route. Each sits on the **first service task reached after** the user task that captures the date, and returns the token to that same user task, so the person corrects the date in the form they already filled in and submits again.

Before this change the validator threw `ValidationException`, which failed the job: the model's three retries were spent on three identical attempts to fix a date that only a person can correct, and the instance then parked on an incident until somebody edited a variable in Operate by hand. Four of the seven faults raised during the v16.0 demonstration had that shape.

The wiring is declared once, in `tools/spec_v3.py` as `DATE_REPAIR_POINTS`, and `check_date_repair_points()` fails the build if:

- a date rule names a form that no user task binds, or
- a date-capturing user task has no `INVALID_DATE` repair point, or
- the boundary is attached to a service task that is **not** the first one reached after that user task.

The third clause matters because a BPMN error with no matching boundary event raises an unhandled-error incident, which is worse than the retry loop it replaced.

3.1 Evidence that it works

Driving the deployed model (definition v20) through the REST API with a deliberately invalid date:

Step	Observed
start the pathway with `referralDate = 2027-01-01` (future)	instance created, `SEC_Task_CheckPack` open
complete `SEC_Task_CheckPack`	`SEC_Auto_ValidateDocuments` raises `INVALID_DATE`
after the boundary fires	**0 incidents**; a **new** `SEC_Task_CheckPack` is `CREATED`; the instance is `ACTIVE`
resubmit with `referralDate = 2026-01-05` (valid)	the task does **not** come back; **0 incidents**; the pathway continues

Reproduce with:

```
cd bpm
./tools/deploy.sh
# then start an instance with the variables above and complete the task twice
```

---

4. The three date classes, and the current counts

Class	Entries	Form	Worker	Fields
`futureDates`	8	picker blocks the past	rejects the past	appointment, treatment start, expected completion, preferred windows
`pastOnlyDates`	9	**nothing**	rejects the future	referral, decision, review, approval, contact and modification dates
`freeDates`	3	nothing	nothing	`attendanceDate`, `consultationDate` on both clinic-letter forms

The `freeDates` class is deliberate and is explained in `docs/17` section 4: the demonstration compresses time, so the appointment sits in the future and whoever fills the form records the attendance or consultation as that appointment day. A "not later than today" check there only manufactured false failures.

---

5. What is still not complete

These are stated rather than left to be discovered:

1. **Two date-capturing user tasks have no check at all.** `ADM_Task_ContactConsultant` (`contactedOn`, `expectedCompletionDate`) and `ADM_Task_ReferHigher` (`referralDate`) are followed only by end events, so no service task ever runs to validate those dates. The rule is inert for them. `DATE_RULES_WITHOUT_A_CHECK` in `tools/spec_v3.py` names both, so the gap is in the source rather than in someone's memory.

2. **The picker still accepts a future date** on the nine past-only fields. The form states the rule prominently; the worker rejects the value; the token returns to the form. It is recoverable, not impossible.
3. **`CLINICAL_REVIEW` on the delay gateway terminates the case.** The form offers three options, but the model routes only `WIDEN_SEARCH` onward; `KEEP_PATIENT_INFORMED` and `CLINICAL_REVIEW` both take the default flow to `End_PatientInformed`. The label of the second option promises a clinical review that the model does not perform. Recorded as a modelling gap, not fixed here.

---

6. Where the pieces are

What	Where
The rule table (single source of truth)	`workers/src/main/resources/date-rules.json`
Server-side enforcement	`workers/.../support/DateRuleValidator.java`
Form-side hints and picker bounds	`model/forms/*.form`; applied by `tools/harden_forms.py`
Boundary wiring and the build check	`tools/spec_v3.py` (`DATE_REPAIR_POINTS`, `check_date_repair_points`)
The form audit	`tools/forms_inventory.py`, `tools/audit_accessibility.py`